



形式语言与自动机初步

Discrete mathematics

南开大学软件学院 陈锐

e-mail: rzchen@nankai.edu.cn



目录

CONTENTS

- ① 形式语言和形式文法
- ② 有穷自动机
- ③ 图灵机



The background of the slide features a stylized illustration. On the left, there is a tall stack of books. On top of the books, a group of small, dark silhouettes of people are standing and cheering. To the right of the books, a large, light blue number '101' is faintly visible in the background. The overall theme suggests a celebration of learning or a milestone in a course.

形式语言与形式文法

- 字符串和形式语言
- 形式文法

汉语

字符:汉字和标点符号

字符集:合法字符的全体

句子:一串汉字和标点符号

语法:形成句子的规则



形式语言

字符

字母表

字符串

形式文法

自然语言：人与人之间交流的基本手段。

如：汉语、英语、俄语、法语、...等

人工语言：主要用于人与计算机之间的交流。

如：程序设计语言

(也有例外，如世界语)

形式语言：研究自然语言和人工语言都必须遵循的一般规律

研究字符串集合及其性质的学科



计算机处理的主要对象

形式语言与自动机理论的 产生与作用

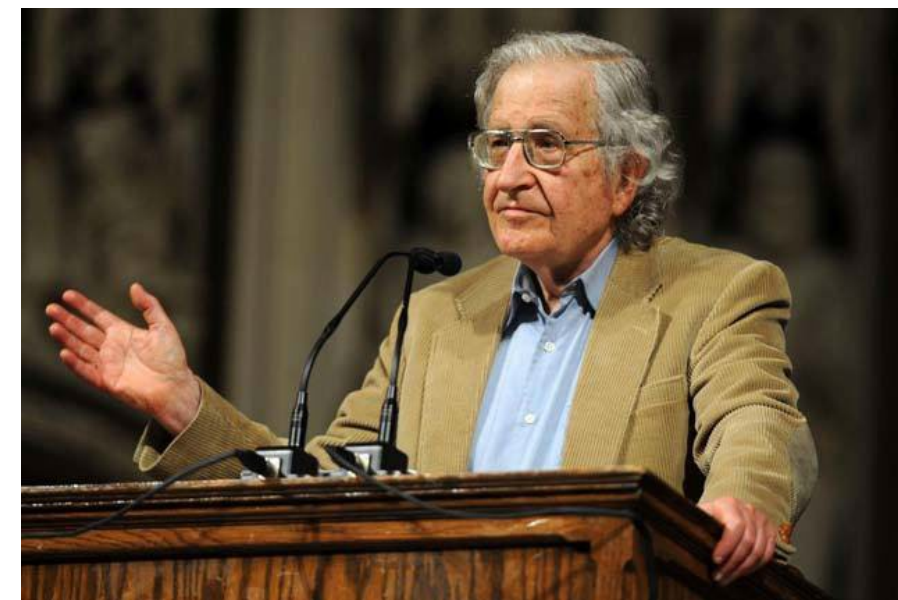


南开大学软件学院
NANKAI UNIVERSITY
COLLEGE OF SOFTWARE

1956, Chomsky从产生语言的角度抽象地将语言形式地定义为由一个字母表中地字母组成的一些串的集合, 组成句子的规则为文法;

1951 - 1956, Kleene从语言识别角度, 给出语言的另一种描述——自动机, 其所识别的句子组成语言; 1959, Chomsky证明文法与自动机的等价性, 此时形式语言诞生。

用于描述程序设计语言文法; 模型化处理; 计算思维培养等。



诺姆·乔姆斯基

字母表 Σ : 非空的有穷集合

字符串: Σ 中符号的有穷序列

如 $\Sigma = \{a, b\}$

$a, b, aab, babb$

字符串 ω 的长度 $|\omega|$: ω 中的字符个数

如 $|a|=1, |aab|=3$

空字符串 ε : 长度为0, 即不含任何符号的字符串

a^n : n 个 a 组成的字符串

Σ^* : Σ 上字符串的全体

字母表: $\Sigma, \Gamma \dots$

字符: $a, b, c \dots$

字符串: $\dots w, x, y, z$

集合: $A, B, C \dots$

子字符串(子串):

字符串中若干连续符号组成的字符串

前缀: 最左端的子串

后缀: 最右端的子串

例如 $\omega = abbaab$

a, ab, abb 是 ω 的前缀

aab, ab, b 是 ω 的后缀

ba 是 ω 的子串, 但既不是前缀, 也不是后缀

ω 本身也是 ω 的子串, 且既是前缀, 也是后缀

ε 也是 ω 的子串, 且既是前缀, 也是后缀



· 字符串的连接运算 ·

设 $\alpha = a_1 a_2 \dots a_n$, $\beta = b_1 b_2 \dots b_m$,

$\alpha\beta = a_1 a_2 \dots a_n b_1 b_2 \dots b_m$ 称作 α 与 β 作的**连接**

如 $\alpha = ab$, $\beta = baa$, $\alpha\beta = abbaa$, $\beta\alpha = baaab$

对任意的字符串 α, β, γ

$$(1) (\alpha\beta)\gamma = \alpha(\beta\gamma)$$

即, 连接运算满足结合律

$$(2) \varepsilon\alpha = \alpha\varepsilon = \alpha$$

即, 空串 ε 是连接运算的单位元

n 个 α 的连接记作 α^n , 串的**幂运算**

如 $(ab)^3 = ababab$, $\alpha^0 = \varepsilon$

设 Σ 为一个字母表，用 Σ^* 表示由 Σ 上的字符组成的全部串（包括空串）的集合；

Σ^+ 则表示 Σ 上的字符组合的所有非空串的集合，即 $\Sigma^+ = \Sigma^* - \{\varepsilon\}$

定义： Σ^* 的子集称作字母表 Σ 上的**形式语言**，简称**语言**

例如 $\Sigma = \{a, b\}$

$$A = \{a, b, aa, bb\}$$

$$B = \{a^n \mid n \in \mathbb{N}\}$$

$$C = \{a^n b^m \mid n, m \geq 1\}$$

$$D = \{\varepsilon\}$$

空语言 $\emptyset$

- 标识符只能由 "数字、字母、下划线_组成，不能由其他符号组成；
- 不能数字开头；
- 严格区分大小写；

**<标识符> : <字母> | <下划线> | <标识符> <字母> |
<标识符> <下划线> | <标识符> <数字>**

<字母> : a | b | ... | z | A | B | ... | Z

<下划线> : _

<数字> : 0 | 1 | ... | 9

定义 形式文法是一个有序4元组 $G = \langle V, T, S, P \rangle$,

其中

- (1) V 是非空有穷集合, V 的元素称作**变元或非终极符**
- (2) T 是非空有穷集合且 $V \cap T = \emptyset$, T 的元素称作**终极符**
- (3) $S \in V$ 称作**起始符**
- (4) P 是非空有穷集合, P 的元素称作**产生式或改写规则**, 形如 $\alpha \rightarrow \beta$, 其中 $\alpha, \beta \in (V \cup T)^*$ 且 $\alpha \neq \varepsilon$.

设文法 $G = \langle V, T, S, P \rangle$, $\omega, \lambda \in (V \cup T)^*$,
 $\omega \Rightarrow \lambda$: 存在 $\alpha \rightarrow \beta \in P$ 和 $\xi, \eta \in (V \cup T)^*$, 使得

$$\omega = \xi \alpha \eta, \quad \lambda = \xi \beta \eta$$

称 ω **直接派生** 出 λ .

$\omega \Rightarrow^* \lambda$: 存在 $\omega_1, \omega_2, \dots, \omega_m$, 使得

$$\omega = \omega_1 \Rightarrow \omega_2 \Rightarrow \dots \Rightarrow \omega_m = \lambda$$

称 ω **派生** 出 λ .

恒有 $\omega \Rightarrow^* \omega$ (当 $m=1$ 时)

$\Rightarrow^*$ 是 $\Rightarrow$ 的自反传递闭包

定义 设文法 $G = \langle V, T, S, P \rangle$, **G 生成的语言**

$$L(G) = \{\omega \in T^* \mid S \xRightarrow{*} \omega\}$$

$L(G)$ 由所有满足下述条件的字符串组成:

- (1) 仅含终结符;
- (2) 可由起始符派生出来.

定义 如果 $L(G_1) = L(G_2)$, 则称文法 G_1 与 G_2 **等价**.

- ※ $L_1 = \{x \in \{0,1\}^* \mid |x| \leq 3\}$ 是字母表 $\{0,1\}$ 上的一个语言。
的元素都是由0和1组成的串，这些串要满足的条件是：
长度不超过3。易知，这样的串有：

$\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111$
共15个。

- ※ $L_2 = \{w \in \{a,b\}^* \mid w = w^R\}$ 也是一个语言。
 L_2 的字母表是 $\{a,b\}$ ，即 L_2 中的元素都是由字符 a 和 b 组成
的串（包括空串）。这些串要满足的条件是 $w = w^R$ ，
即每个串都要同它的逆串相等。这种串称为**回文**
(*palindrome*)

$G_1 = (V_1, T_1, S, P_1)$ $V_1 = \{S, A\}$ $T_1 = \{0, 1\}$

$P_1: S \rightarrow 0S \quad S \rightarrow 0A \quad (S \rightarrow 0S \mid 0A)$

$A \rightarrow 1A \quad A \rightarrow \varepsilon \quad (A \rightarrow 1A \mid \varepsilon)$

上面给出了一个文法 G ，它的变量集、终极符集、产生式和初始符都已经明确给出。下面求解它产生的语言。

第1步：通过推导来得到 $L(G)$ 中的一些句子

$S \Rightarrow 0S \Rightarrow 00S \Rightarrow 000A \Rightarrow 0001A \Rightarrow 0001$

$S \Rightarrow 0A \Rightarrow 01A \Rightarrow 01$

$S \Rightarrow 0A \Rightarrow 0$

$S \Rightarrow 0S \Rightarrow 00A \Rightarrow 001A \Rightarrow 0011A \Rightarrow 00111A \Rightarrow 00111$

第2步：归纳出 $L(G_1)$ 形式表示

由于 $L(G_1)$ 中的元素都是终极符串，只能由终极符0和1组成，因此每一个推导过程都最终要把变量 S 和 A 用0和1替换。右端不含 S 和 A 的产生式只有一个： $A \rightarrow \varepsilon$ 。

因此，每一个推导过程中，都必须有的一步用到产生式把 S 换成 A 。

$L(G_1)$ 中的句子必有 的形式，其中 。这样就可以写出 $L(G_1)$ 的形式表示：

$$L(G_1) = \{0^i 1^j \mid i \geq 1, j \geq 0\}$$

第3步，证明上一步归纳出的形式表示的正确性

从两个方面来证明

- 1) 凡是根据 G_1 能推导出来的句子都包含在 $L(G_1)$ 的形式表示中；
- 2) 每个 $L(G_1)$ 中的终极字符串都可以从 G_1 推导出来。

例1 文法 $G_1 = \langle V, T, S, P \rangle$, **其中** $V = \{S\}$, $T = \{a, b\}$,

$P: S \rightarrow aSb \mid ab$

$L(G_1) = \{a^n b^n \mid n > 0\}$

例2 文法 $G_2 = \langle V, T, S, P \rangle$, **其中** $V = \{A, B, S\}$, $T = \{0, 1\}$,

$P: S \rightarrow 1A, A \rightarrow 0A \mid 1A \mid 0B, B \rightarrow 0$

$L(G_2) = \{1x00 \mid x \in \{0, 1\}^*\}$

例3 文法 $G_3 = \langle V, T, S, P \rangle$, **其中** $V = \{A, B, S\}$, $T = \{0, 1\}$,

$P: S \rightarrow B0, B \rightarrow A0, A \rightarrow A1 \mid A0, A \rightarrow 1$

$L(G_3) = L(G_2)$, G_3 与 G_2 等价

例4 $G = \langle V, T, S, P \rangle$, 其中 $V = \{S, A, B, C, D, E\}$, $T = \{a\}$,

P : (1) $S \rightarrow ACaB$ (2) $Ca \rightarrow aaC$ (3) $CB \rightarrow DB$ (4) $CB \rightarrow E$
 (5) $aD \rightarrow Da$ (6) $AD \rightarrow AC$ (7) $aE \rightarrow Ea$ (8) $AE \rightarrow \varepsilon$

试证明: $\forall i \geq 1, S \xRightarrow{*} a^{2^i}$

证: a^2 和 a^4 的派生过程

$$S \Rightarrow ACaB \quad (1)$$

$$\Rightarrow AaaCB \quad (2)$$

$$\Rightarrow AaaE \quad (4)$$

$$\xRightarrow{*} \Rightarrow AEaa \quad \text{2次 (7)}$$

$$a^2 \quad (8)$$

$S \xRightarrow{*} AaaCB$

$\Rightarrow AaaDB \quad (3)$

$\xRightarrow{*} ADaaB \quad 2\text{次}(5)$

$\Rightarrow ACaaB \quad (6)$

$\xRightarrow{*} AaaaaCB \quad 2\text{次}(2)$

$\Rightarrow AaaaaE \quad (4)$

$\xRightarrow{*} AEaaaa \quad 4\text{次}(7)$

$\Rightarrow a^4 \quad (8)$

于是, $\forall i \geq 1$,

$$\begin{aligned} S &\stackrel{*}{\Rightarrow} Aa^{2^i}CB \\ &\Rightarrow Aa^{2^i}E \end{aligned} \quad (4)$$

$$\stackrel{*}{\Rightarrow} AEa^{2^i} \quad 2^i \text{ 次} (7)$$

$$\Rightarrow a^{2^i} \quad (8)$$

可以证明:

$$L(G) = \{ a^{2^i} \mid i \geq 1 \}$$

Chomsky谱系

0型文法(短语结构文法,无限制文法)

1型文法(上下文有关文法):

所有产生式 $\alpha \rightarrow \beta$, 满足 $|\alpha| \leq |\beta|$

另一个等价的定义: 所有的产生式形如

$$\xi A \eta \rightarrow \xi \alpha \eta$$

其中 $A \in V$, $\xi, \eta, \alpha \in (V \cup T)^*$, 且 $\alpha \neq \varepsilon$

2型文法(上下文无关文法):

所有的产生式形如 $A \rightarrow \alpha$

其中 $A \in V, \alpha \in (V \cup T)^*$,

3型文法(正则文法): 右线性文法和左线性文法的统称

右线性文法: 所有的产生式形如

$$A \rightarrow \alpha B \text{ 或 } A \rightarrow \alpha$$

左线性文法: 所有的产生式形如

$$A \rightarrow B\alpha \text{ 或 } A \rightarrow \alpha$$

其中 $A, B \in V, \alpha \in T^*$

例1是上下文无关文法

例2是右线性文法,例3是左线性文法,都是正则文法

例4是0型文法

0型语言: 0 型文法生成的语言

1型语言(上下文有关语言): 如果 $L - \{\varepsilon\}$ 可由1型文法生成, 则称 L 是1型语言

2型语言(上下文无关语言): 2 型文法生成的语言

3型语言(正则语言): 3 型文法生成的语言

如 $\{1x00 \mid x \in \{0, 1\}^*\}$ 是正则语言 (例1)

$\{a^n b^n \mid n > 0\}$ 是上下文无关语言 (例2,3)

$\{a^{2^i} \mid i \geq 1\}$ 是 0 型语言 (例4)

定理 0型语言 $\supset$ 1型语言 $\supset$ 2型语言 $\supset$ 3型语言

$$G = \{\{E, T, F\}, \{a, +, -, *, /, (,)\}, E, P\}$$

其中 E : 算术表达式, T : 项,

F : 因子, a : 数或变量

$$P: E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid a$$

这是上下文无关文法

定理 设 G 是右(左)线性文法,则存在左(右)线性文法 G' 使得 $L(G')=L(G)$.

证明: G' 用模拟 G

$$G = \langle V, T, S, P \rangle$$

$$P: A \rightarrow \alpha B$$

$$A \rightarrow \alpha$$

$$G' = \langle V \cup \{S'\}, T, S', P' \rangle$$

$$P': B \rightarrow A\alpha$$

$$S' \rightarrow A\alpha$$

$$S \rightarrow \varepsilon$$

模拟例2中的 G_2

$$G_2 = \langle V, T, S, P \rangle$$

$$V = \{A, B, S\}$$

$$T = \{0, 1\}$$

$$P: S \rightarrow 1A$$

$$A \rightarrow 0A$$

$$A \rightarrow 1A$$

$$A \rightarrow 0B$$

$$B \rightarrow 0$$

$$G_2' = \langle V', T', S', P' \rangle$$

$$V' = \{A, B, S, S'\}$$

$$T' = \{0, 1\}$$

$$P': A \rightarrow S1$$

$$A \rightarrow A0$$

$$A \rightarrow A1$$

$$B \rightarrow A0$$

$$S' \rightarrow B0$$

$$S \rightarrow \epsilon$$

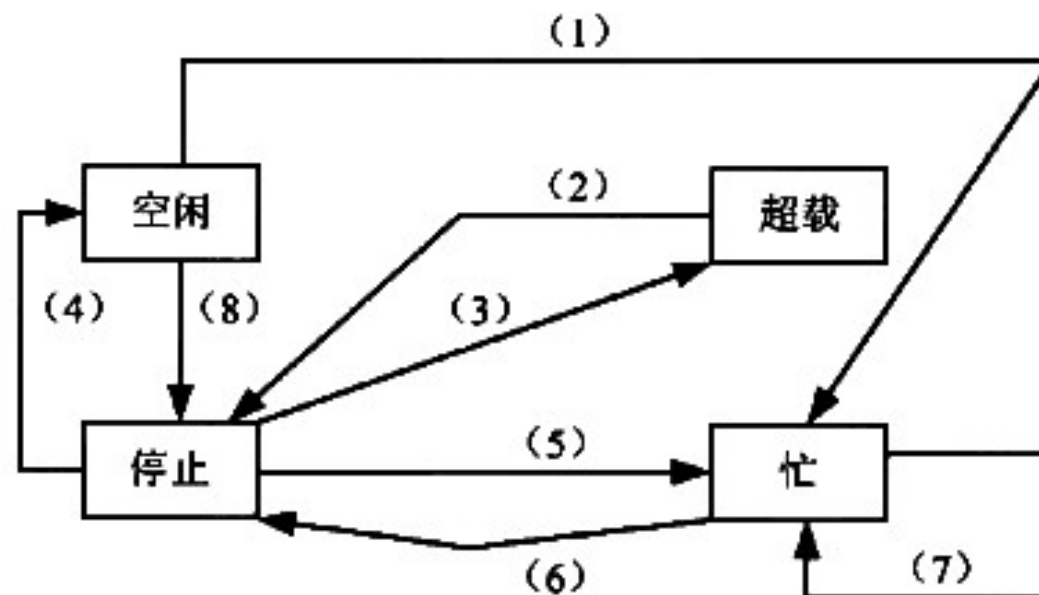
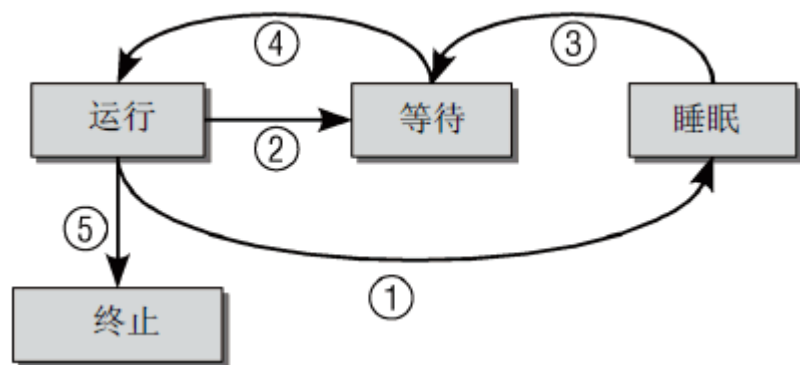
可删去 G_2' 中的 S , 这实际上就是例3中的 G_3

有穷自动机

- 确定型有穷自动机(DFA)
- 非确定型有穷自动机(NFA)
- 带 ϵ 转移的NFA(ϵ -NFA)



- 有限状态机: Moore Machine, Mealy Machine
- 数字电路设计
- 电脑游戏的AI 设计
- 各种通讯协议: TCP, HTTP, Bluetooth, Wifi
- 文本搜索, 词法分析



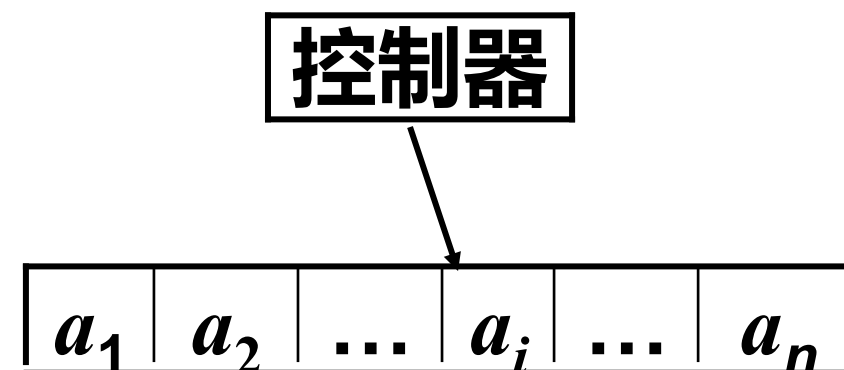


· 确定型有穷自动机 ·

定义 确定型有穷自动机(DFA)是一个有序5元组

$M = \langle Q, \Sigma, \delta, q_0, F \rangle$, 其中

- (1) **状态集合** Q : 非空有穷集合
- (2) **输入字母表** Σ : 非空有穷集合
- (3) **状态转移函数** $\delta: Q \times \Sigma \rightarrow Q$
- (4) **初始状态** $q_0 \in Q$
- (5) **终结状态集** $F \subseteq Q$



例：设计DFA, 在任何由0 和1 构成的串中, 接受含有01 子串的全部串.

- ① 未发现01, 即使0 都还没出现过;
- ② 未发现01, 但刚刚读入字符是0;
- ③ 已经发现了01.

可以设 $M = (\{q_1, q_2, q_3\}, \{0, 1\}, \delta, q_1, \{q_3\})$

$$\begin{aligned}\delta(q_1, 1) &= q_1 & \delta(q_2, 1) &= q_3 & \delta(q_3, 1) &= q_3 \\ \delta(q_1, 0) &= q_2 & \delta(q_2, 0) &= q_2 & \delta(q_3, 0) &= q_3\end{aligned}$$

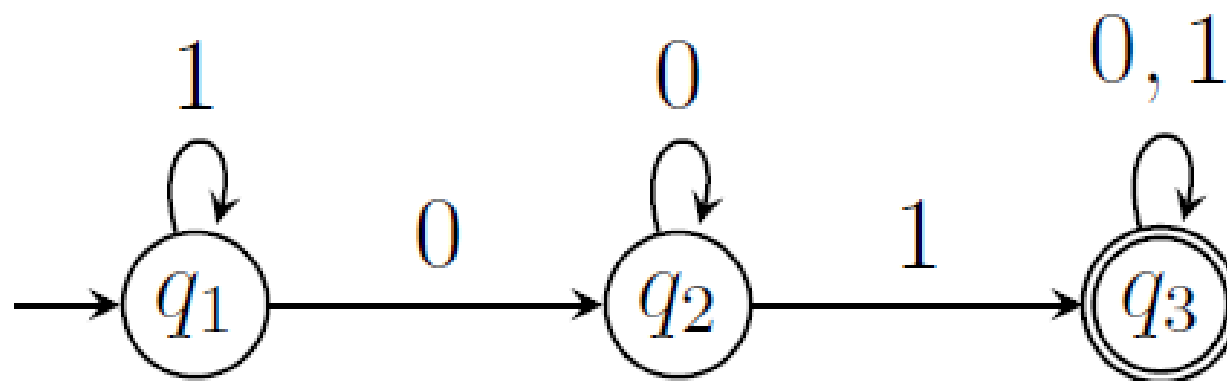
例：设计DFA, 在任何由0 和1 构成的串中, 接受含有01 子串的全部串.

- ① 未发现01, 即使0 都还没出现过;
- ② 未发现01, 但刚刚读入字符是0;
- ③ 已经发现了01.

可以设 $M = (\{q_1, q_2, q_3\}, \{0, 1\}, \delta, q_1, \{q_3\})$

$$\begin{aligned}\delta(q_1, 1) &= q_1 & \delta(q_2, 1) &= q_3 & \delta(q_3, 1) &= q_3 \\ \delta(q_1, 0) &= q_2 & \delta(q_2, 0) &= q_2 & \delta(q_3, 0) &= q_3\end{aligned}$$

- 每个状态 q 对应一个节点, 用圆圈表示;
- 状态转移 $\delta(q, a) = p$ 为一条从 q 到 p 且标记为字符 a 的有向边;
- 开始状态 q_0 用一个标有start 的箭头表示;
- 接受 (终结) 状态的节点, 用双圆圈表示.



- 每个状态 q 对应一行, 每个字符 a 对应一列;
- 若有 $\delta(q, a) = p$, 用第 q 行第 a 列中填入的 p 表示;
- 开始状态 q_0 前, 标记箭头 $\rightarrow$ 表示;
- 接受状态 $q \in F$ 前, 标记星号 $*$ 表示.

δ	0	1
$\rightarrow q_1$	q_2	q_1
q_2	q_2	q_3
$*q_3$	q_3	q_3

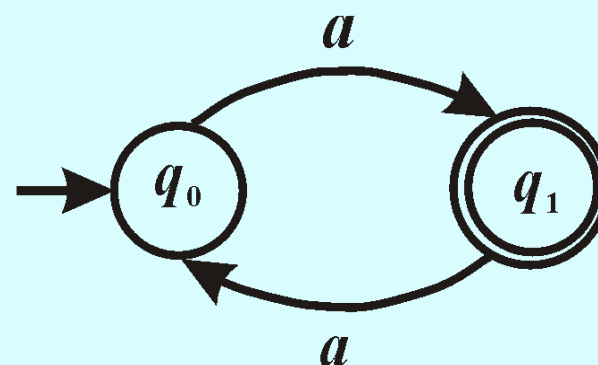
例 $M = \langle \{q_0, q_1\}, \{a\}, \delta, q_0, \{q_1\} \rangle$

$$\delta(q_0, a) = q_1, \delta(q_1, a) = q_0$$

$$\delta^*(q_0, a^n) = \begin{cases} q_1, & n \text{ 为奇数} \\ q_0, & n \text{ 为偶数} \end{cases}$$

$$\delta^*(q_1, a^n) = \begin{cases} q_0, & n \text{ 为奇数} \\ q_1, & n \text{ 为偶数} \end{cases}$$

$$L(M) = \{a^{2k+1} \mid k \in \mathbb{N}\}$$



- 如果语言 L 是某个DFA M 的语言, 即 $L = L(M)$, 则称 L 是正则语言.
- $\emptyset, \{\varepsilon\}$ 都是正则语言
- 若 Σ 是字母表, Σ^*, Σ^n 都是 Σ 上的正则语言

例 文法 $G = \langle V, T, S, P \rangle$, 其中 $V = \{A, B, S\}$, $T = \{0, 1\}$,
 $P: S \rightarrow 1A, A \rightarrow 0A \mid 1A \mid 0B, B \rightarrow 0$
 $L(G) = \{1x00 \mid x \in \{0, 1\}^*\}$



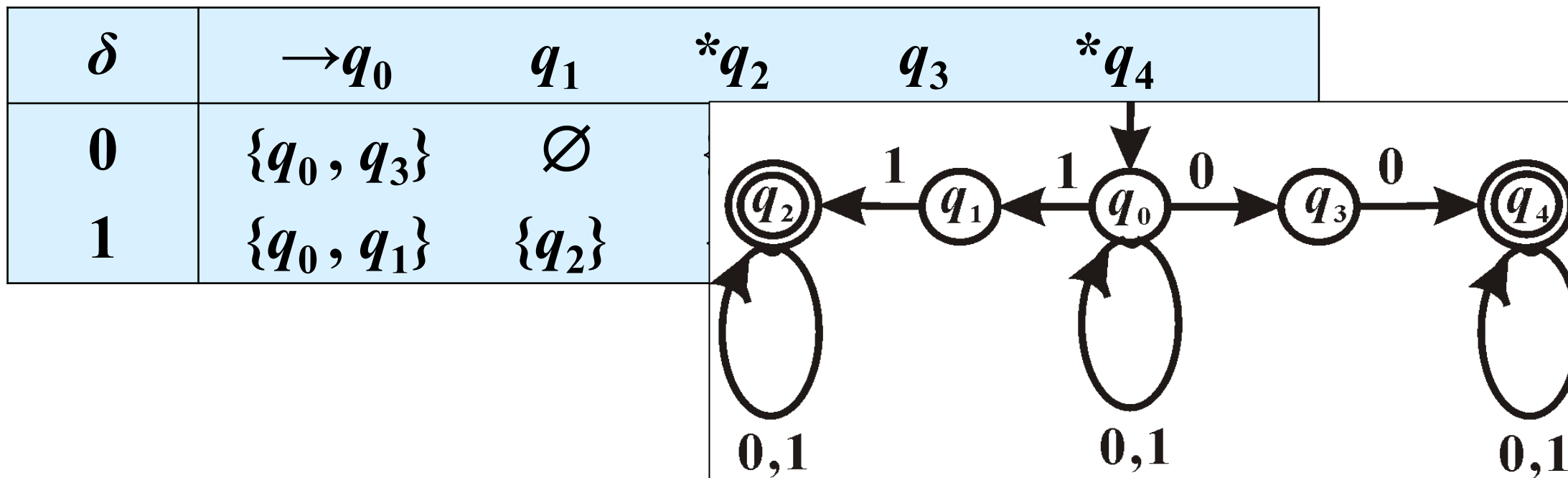
·非确定型有穷自动机·

定义 非确定型有穷自动机 (NFA)

$$M = \langle Q, \Sigma, \delta, q_0, F \rangle ,$$

其中 Q, Σ, q_0, F 的定义与 DFA 的相同, 而

$$\delta: Q \times \Sigma \rightarrow P(Q)$$



$\delta^*: Q \times \Sigma^* \rightarrow Q$ 递归定义如下: $\forall q \in Q, a \in \Sigma$ 和 $w \in \Sigma^*$

$$\delta^*(q, \varepsilon) = \{q\}$$

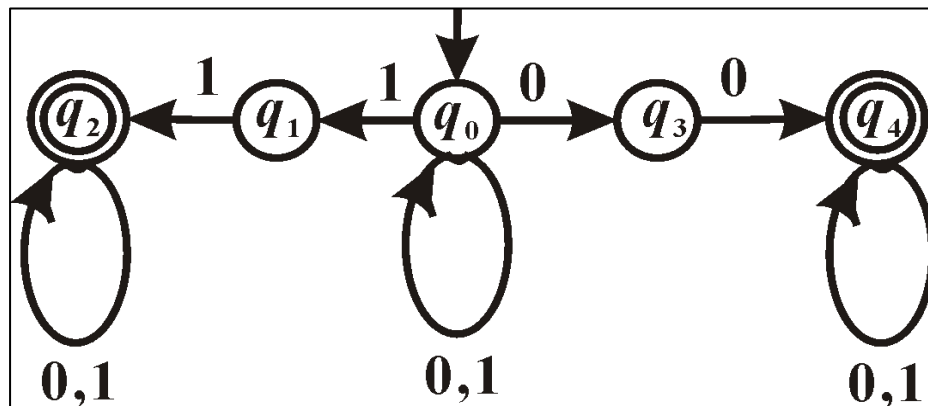
$$\delta^*(q, wa) = \bigcup_{p \in \delta^*(q, w)} \delta(p, a)$$

定义 $\forall w \in \Sigma^*$, 如果 $\delta^*(q_0, w) \cap F \neq \emptyset$, 则称 **M 接受 w** . M 接受的字符串的全体称作 **M 接受的语言**, 记作 $L(M)$, 即

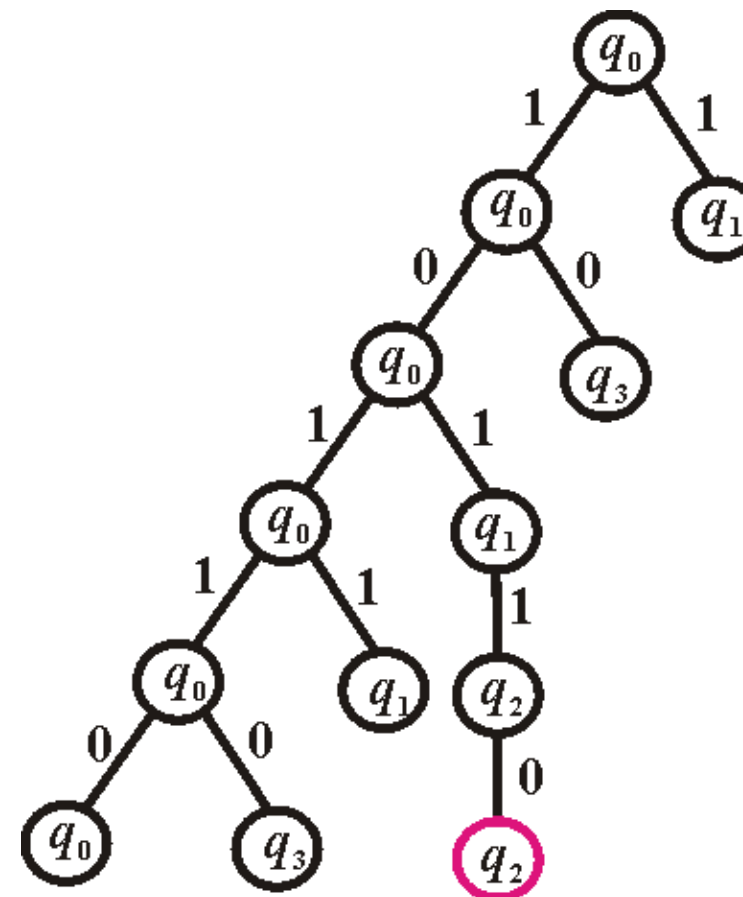
$$L(M) = \{ w \in \Sigma^* \mid \delta^*(q_0, w) \cap F \neq \emptyset \}$$



NFA接受的语言



w	$\delta^*(q_0, w)$
1	$\{q_0, q_1\}$
10	$\{q_0, q_3\}$
101	$\{q_0, q_1\}$
1011	$\{q_0, q_1, q_2\}$
10110	$\{q_0, q_2, q_3\}$



$$L(G) = \{ x00y, x11y \mid x, y \in \{0,1\}^* \}$$



定理 对每一个NFA M 都存在DFA M' 使得 $L(M)=L(M')$

用 $M'=\langle Q',\Sigma,\delta',q_0',F' \rangle$ 模拟 $M=\langle Q,\Sigma,\delta,q_0,F \rangle$

$$Q'=P(Q), q_0'=\{q_0\}$$

$$F'=\{ A \in Q \mid A \cap F \neq \emptyset \}$$

$\forall A \in Q$ 和 $a \in \Sigma$,

$$\delta'(A, a) = \bigcup_{p \in A} \delta(p, a)$$

证明: 为证明 $L(M)=L(M')$ 对 $|w|$ 用归纳法,

先证明: $\delta'(\{q_0\}, w) = \delta(q_0, w)$

当 $|w| = 0$ 即 $w = \varepsilon$,

$$\delta'(\{q_0\}, \varepsilon) = \{q_0\} = \delta(q_0, \varepsilon)$$

假设 $|w| = n$ 时命题成立,

归纳递推 $|w| = n + 1$ 时, $w = xa$ ($a \in \Sigma$)

$\delta(q_0, xa) = \bigcup_{p \in \delta(q_0, x)} \delta(p, a)$ NFA 中 δ 定义

$= \bigcup_{p \in \delta'(\{q_0\}, x)} \delta(p, a)$ 归纳假设

$= \delta''(\delta'(\{q_0\}, x), a)$ D的构造

$= \delta'(\{q_0\}, xa)$ DFA中 δ 定义

由于 $\delta'(\{q_0\}, w) = \delta(q_0, w)$

$$w \in L(M) \leftrightarrow \delta(q_0, w) \cap F \neq \emptyset$$

$$\leftrightarrow \delta'(\{q_0\}, w) \cap F \neq \emptyset$$

$$\leftrightarrow \delta'(\{q_0\}, w) \in F'$$

$$\leftrightarrow w \in L(M')$$

$$F' = \{ A \in Q \mid A \cap F \neq \emptyset \}$$

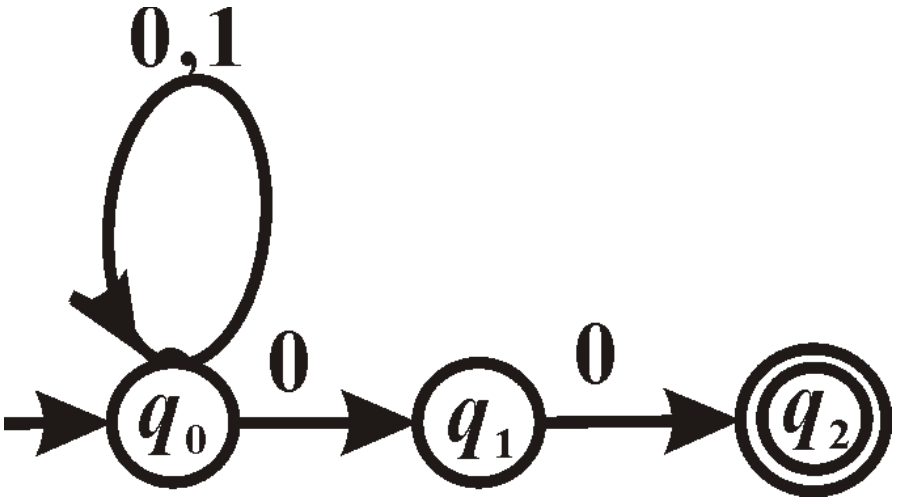
定义 $\forall w \in \Sigma^*$, 如果 $\delta^*(q_0, w) \cap F \neq \emptyset$, 则称 **M 接受 w** . M 接受的字符串的全体称作 **M 接受的语言**, 记作 $L(M)$, 即

$$L(M) = \{ w \in \Sigma^* \mid \delta^*(q_0, w) \cap F \neq \emptyset \}$$



NFA M

δ	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\emptyset$
$*q_2$	$\emptyset$	$\emptyset$



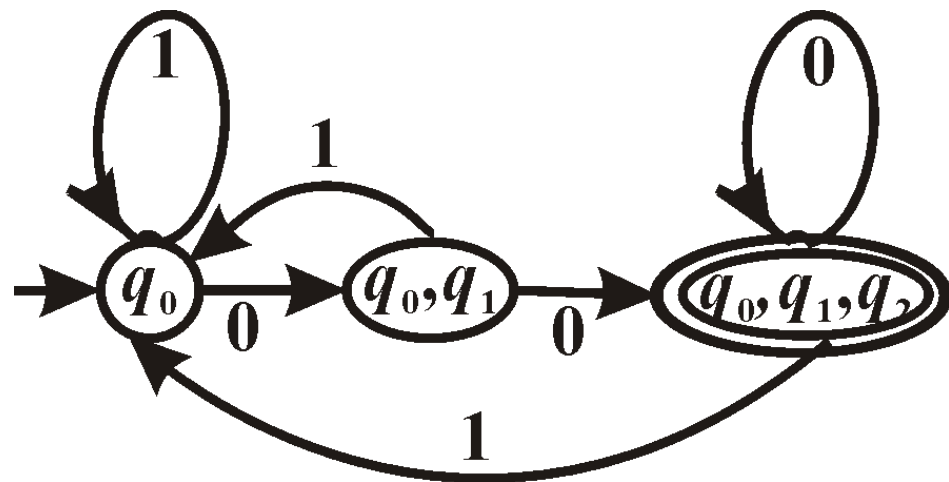
DFA M'

δ'	0	1
$\rightarrow\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\{q_2\}$	$\emptyset$
$*\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$*\{q_1, q_2\}$	$\{q_2\}$	$\emptyset$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$
$\emptyset$	$\emptyset$	$\emptyset$

不可达状态:从初始状态出发永远不可能达到的状态删去所有的不可达状态, 不会改变FA接受的语言.

如 M' 中的 $\{q_1\}, \{q_2\}, \{q_0, q_2\}, \{q_1, q_2\}$ 和 $\emptyset$ 都是不可达状态, 删去这些状态得到 M''

δ''	0	1
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$



ε 转移: 不读任何符号, 自动转移状态.

ε -NFA: $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$

定理 对每一个 ε -NFA M 都存在 DFA M' 使得

$$L(M) = L(M')$$

DFA, NFA 和 ε -NFA 接受同一个语言类

- 自动机在某状态, 读入某个字符时, 可能有多多个转移;
- 自动机在某状态, 读入某个字符时, 可能没有转移;
- 自动机在某状态, 可能不读入字符, 就进行转移.

定义: 状态 q 的 ε -闭包(ε -closure), 记为 $\text{Eclose}(q)$ 或 $E(q)$, 表示从 q 经过 $\varepsilon\varepsilon \cdots \varepsilon$ 序列可达的全部状态集合, 递归定义为:

- 1 $q \in \text{Eclose}(q)$;
- 2 $\forall p \in \text{Eclose}(q)$, 若 $r \in \delta(p, \varepsilon)$, 则 $r \in \text{Eclose}(q)$.

模拟的方法与用DFA模拟不带 ε 的NFA的方法基本相同, 只是要用 ε -closure(q)代替 q .

用DFA $M' = \langle Q', \Sigma, \delta', q_0', F' \rangle$ 模拟 ε -NFA $M = \langle Q, \Sigma, \delta, q_0, F \rangle$

$$Q' = P(Q), q_0' = \varepsilon\text{-closure}(q_0)$$

$$F' = \{A \in Q \mid A \cap F \neq \emptyset\}$$

$\forall A \in Q$ 和 $a \in \Sigma$,

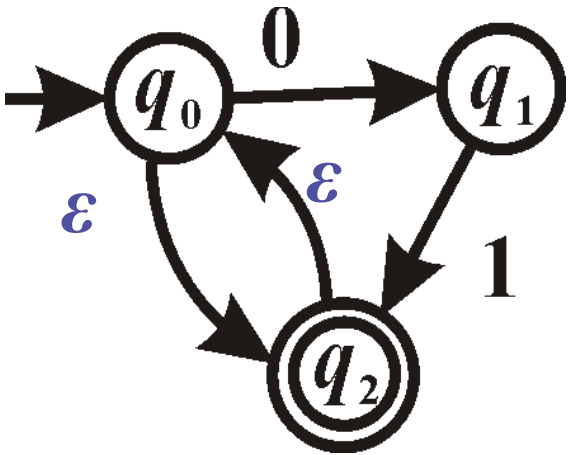
$$\delta'(A, a) = \bigcup_{q \in A} \{r \in E(t) \mid \exists p, t \text{ 使 } p \in E(q), t \in \delta(p, a)\}$$

构造DFA M' 时不需要对不可达状态进行计算, 做法如下: 从 $q_0' = E(q_0)$ 开始, 对每一个 $a \in \Sigma$ 计算 δ' 的值, 然后对每一个新出现的子集计算 δ' 的值, 重复进行, 直至没有新的子集出现为止.



ε -NFA M

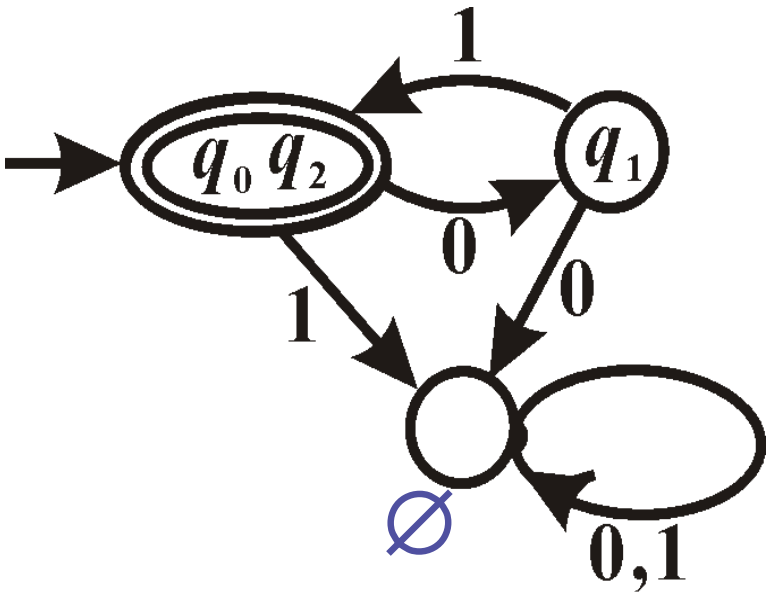
δ	0	1	ε
$\rightarrow q_0$	$\{q_1\}$	$\emptyset$	$\{q_0, q_2\}$
q_1	$\emptyset$	$\{q_2\}$	$\{q_1\}$
$*q_2$	$\emptyset$	$\emptyset$	$\{q_0, q_2\}$



$L(M)=L(M')=\{ (01)^n \mid n \geq 0 \}$

DFA M'

δ'	0	1
$\rightarrow^* \{q_0, q_2\}$	$\{q_1\}$	$\emptyset$
$\{q_1\}$	$\emptyset$	$\{q_0, q_2\}$
$\emptyset$	$\emptyset$	$\emptyset$



有穷自动机

通过**机器**装置描述正则语言

用计算机编写相应算法, 易于实现

正则表达式

通过**表达式**描述正则语言, 代数表示方法, 使用方便

应用广泛

grep 工具(*Global Regular Expression and Print*)

Emacs / Vim 文本编辑器

lex / flex 词法分析器

各种程序设计语言 *Python / Perl / Haskell / ...*

设 L 和 M 是两个语言, 那么

并 $L \cup M = \{w \mid w \in L \text{ 或 } w \in M\}$

连接 $L \cdot M = \{w \mid w = xy, x \in L \text{ 且 } y \in M\}$

幂 $L_0 = \{\varepsilon\}$

$L_1 = L$

$L_n = L_{n-1} \cdot L$

克林闭包 $L^* = \bigcup_{i=0}^{\infty} L^i$

如果 Σ 为字母表, 则 Σ 上的正则表达式递归定义为:

- ① $\emptyset$ 是一个正则表达式, 表示空语言;
- ② ε 是一个正则表达式, 表示语言 $\{\varepsilon\}$;
- ③ $\forall a \in \Sigma, a$ 是一个正则表达式, 表示语言 $\{a\}$;
- ④ 如果正则表达式 r 和 s 分别表示语言 R 和 S , 那么

$r + s, rs, r^*$ 和 (r)

都是正则表达式, 分别表示语言:

$R \cup S, R \cdot S, R^*$ 和 R .

定理 设 G 是右线性文法, 则存在 ε -NFA M 使得 $L(M)=L(G)$; 设 M 是DFA, 则存在右线性文法 G 使得 $L(G)=L(M)$.

定理 下述命题是等价的:

- (1) L 是正则语言;
- (2) 语言 L 能由右线性文法生成;
- (3) 语言 L 能由左线性文法生成;
- (4) 语言 L 能被DFA接受;
- (5) 语言 L 能被NFA接受;
- (6) 语言 L 能被 ε -NFA接受.

设右线性文法 $G = \langle V, T, S, P \rangle$

ε -NFA $M = \langle Q, \Sigma, \delta, q_0, F \rangle$ 构造如下:

$$Q = V \cup \{q_f\}, \quad q_0 = \{S\}, \quad F = \{q_f\},$$

$$\Sigma = \{ \alpha \in T^* - \{\varepsilon\} \mid \text{存在 } A \rightarrow \alpha B \in P \text{ 或 } A \rightarrow \alpha \in P \}$$

$\forall A \in V$ 和 $\alpha \in \Sigma \cup \{\varepsilon\}$,

若 $A \rightarrow \alpha B \in P$, 则 $\delta(A, \alpha)$ 中含有 B ;

若 $A \rightarrow \alpha \in P$, 则 $\delta(A, \alpha)$ 中含有 q_f ;

$$\forall \alpha \in \Sigma \cup \{\varepsilon\}, \quad \delta(q_f, \alpha) = \emptyset$$

$G = \langle V, T, S, P \rangle$

$V = \{A, S\}$

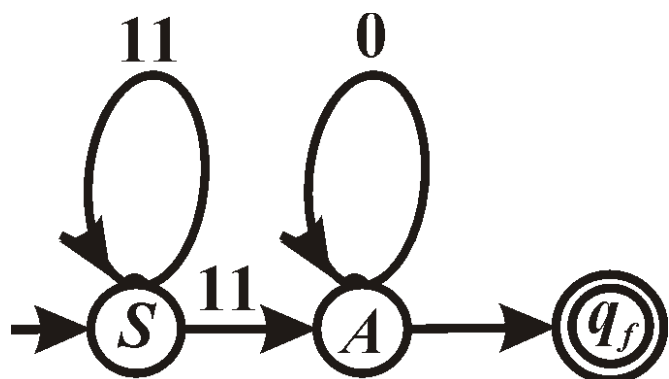
$T = \{0, 1\}$

$P: S \rightarrow 11S$

$S \rightarrow 11A$

$A \rightarrow 0A$

$A \rightarrow \varepsilon$



$\varepsilon\text{-NFA } M = \langle Q, \Sigma, \delta, S, \{q_f\} \rangle$

$Q = \{A, S, q_f\}$

$\Sigma = \{11, 0\}$

δ	11	0	ε
$\rightarrow S$	$\{S, A\}$	$\emptyset$	$\emptyset$
A	$\emptyset$	$\{A\}$	$\{q_f\}$
$*q_f$	$\emptyset$	$\emptyset$	$\emptyset$

$L(G) = L(M) = \{ (11)^m 0^n \mid m \geq 1, n \geq 0 \}$

设DFA $M = \langle Q, \Sigma, \delta, q_0, F \rangle$

右线性文法 $G = \langle V, T, S, P \rangle$ 构造如下:

$$V = Q, \quad T = \Sigma, \quad S = q_0$$

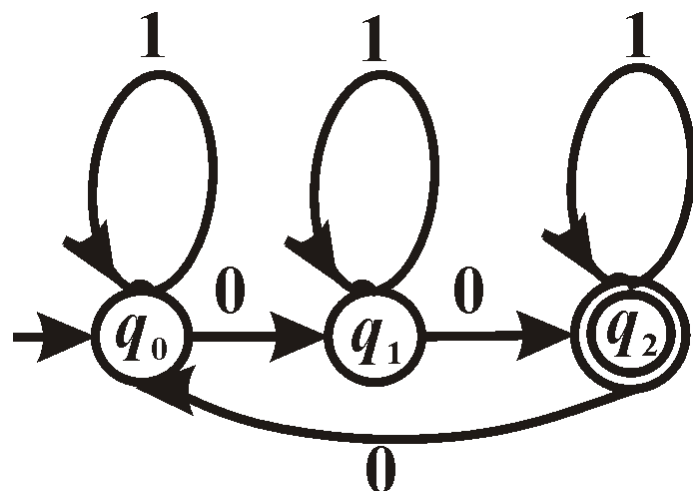
$\forall q \in Q$ 和 $a \in \Sigma$,

若 $\delta(q, a) = p$, 则有产生式 $q \rightarrow ap$

若 $q \in F$, 则有产生式 $q \rightarrow \varepsilon$

DFA M

δ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_1
$*q_2$	q_0	q_2



$G = \langle V, T, S, P \rangle$

$V = \{q_0, q_1, q_2\}, T = \{0, 1\}, S = q_0$

$P: q_0 \rightarrow 0q_1 \quad q_0 \rightarrow 1q_0$

$q_1 \rightarrow 0q_2 \quad q_1 \rightarrow 1q_1$

$q_2 \rightarrow 0q_0 \quad q_2 \rightarrow 1q_2$

$q_2 \rightarrow \varepsilon$

$L(M) = L(G)$, 它们是所有含 $3k+2$ ($k \geq 0$) 个 0 的 0,1 串组成的集合



- 一个系统，接受**输入**，改变自身的**状态**，产生**输出**
- 状态指为了完成机器的任务而对输入序列进行的一种临时归类
- 可以理解为一种对记忆的模拟
- 在逐个接受输入字符的过程中，机器状态会发生**多次**改变，最终会停止在某个状态，并有**输出产生**
- 机器状态的数目有限，就是有限状态机



图灵机

- 图灵机的基本模型
- 图灵机接受的语言
- 用图灵机计算函数



1900年 D. Hilbert 在巴黎第二届数学家大会上提出著名的23个问题.

**第10个问题:如何判定整系数多项式是否有整数根?
要求使用 “有限次运算的过程”**

1970 年证明不存在这样的判定算法, 即这个问题是不可判定的, 或不可计算的.

从20世纪30年代先后提出

图灵机 A.M.Turing, 1936年

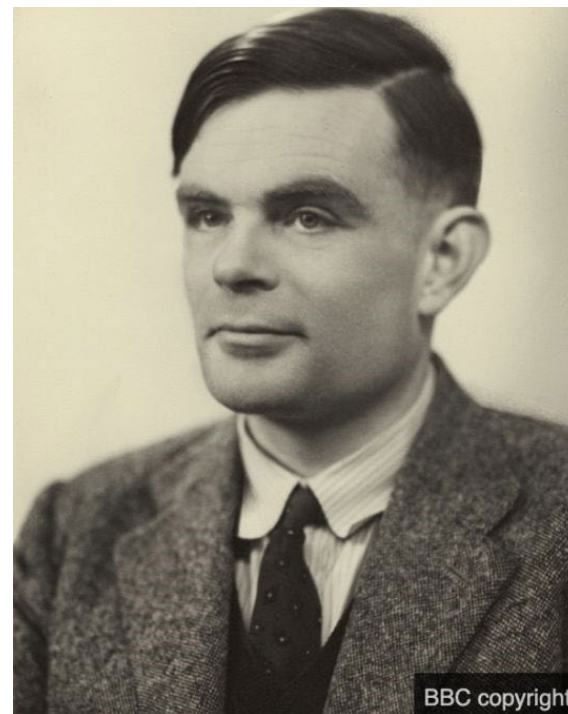
λ 转换演算 A.Church, 1935年

递归函数 K.Gödel, 1936年

正规算法 A.A.Markov, 1951年

无限寄存器机器 J.C.Shepherdson, 1963年

...



Alan Turing

1912 – 1954

1912 Alan Mathison Turing was born in West London

1936 Produced “On Computable Numbers”, aged 24

1952 Convicted of gross indecency for his relationship with a man

2013 Received royal pardon for the conviction

Source: BBC

BBC copyright

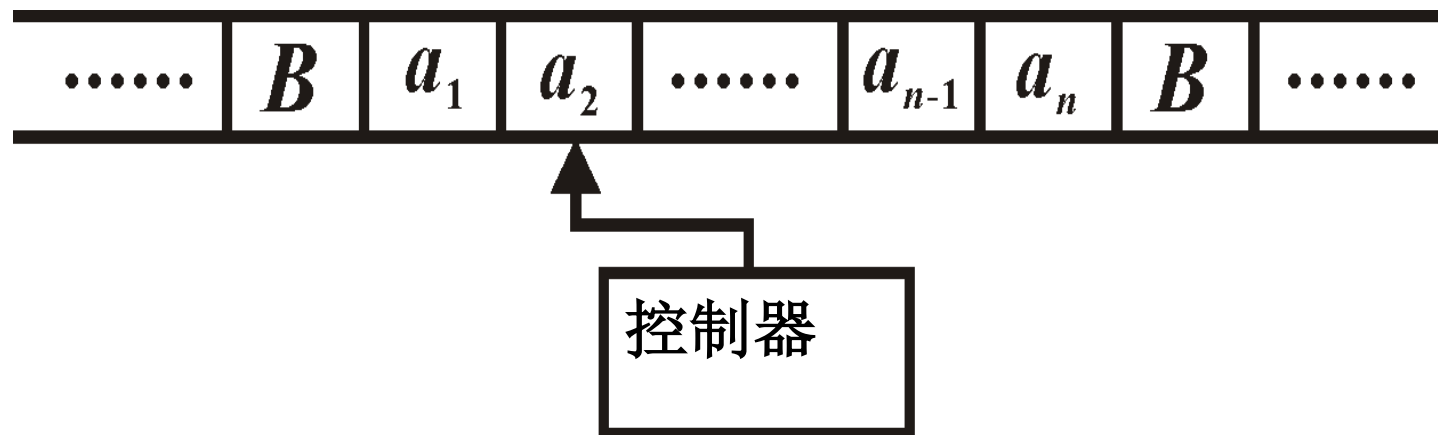


已经证明这些模型都是等价的, 即它们计算的函数类(识别的语言类) 是相同的.

Church-Turing论题: 直观可计算的函数类就是图灵机以及任何与图灵机等价的计算模型可计算 (可定义) 的函数类

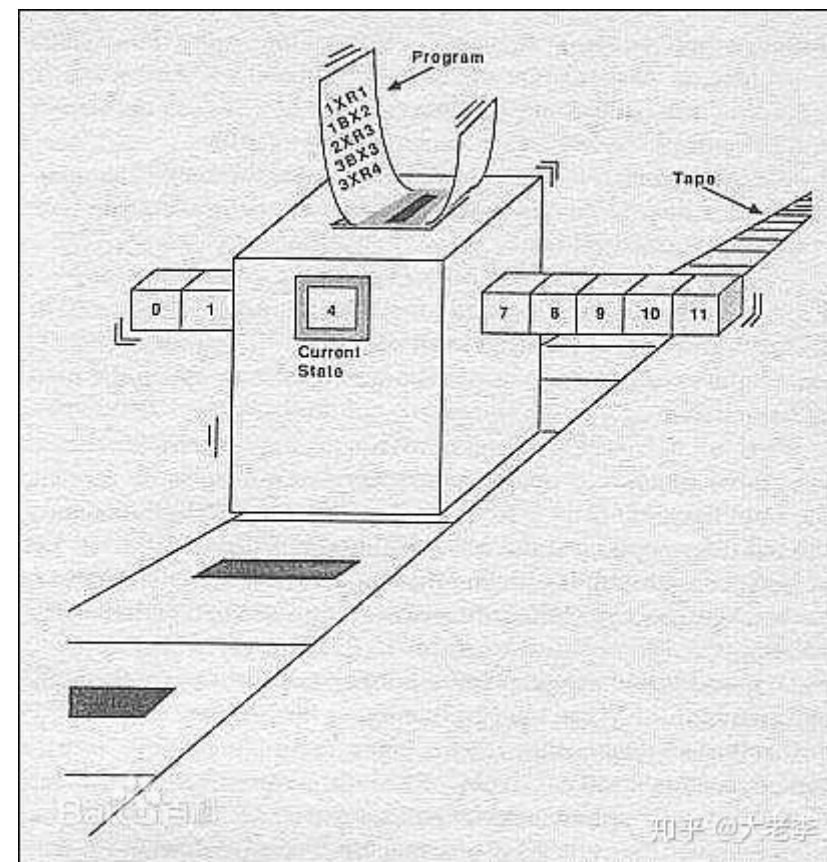


图灵机的基本模型



定义 图灵机(TM) $M = \langle Q, \Sigma, \Gamma, \delta, q_0, B, A \rangle$, 其中

- (1) **状态集合** Q : 非空有穷集合;
- (2) **输入字母表** Σ : 非空有穷集合;
- (3) **带字母表** Γ : 非空有穷集合且 $\Sigma \subset \Gamma$;
- (4) **初始状态** $q_0 \in Q$;



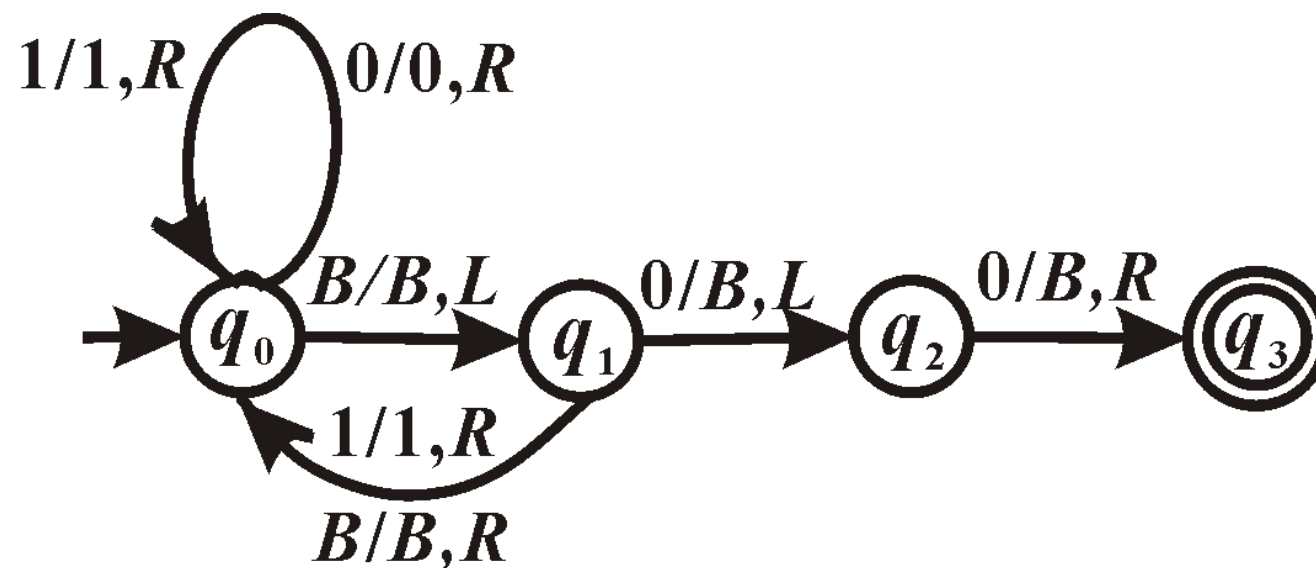
- (5) **空白符** $B \in \Gamma - \Sigma$;
- (6) **接受状态集** $A \subseteq Q$;
- (7) **动作函数** δ 是 $Q \times \Gamma$ 到 $\Gamma \times \{L, R\} \times Q$ 的部分函数,
即 $\text{dom} \delta \subseteq Q \times \Gamma$.

$\delta(q, s) = (s', R, q')$ 的含义: 当处于状态 q , 读写头扫视符号 s 时, M 的下一步把状态转移到 q' , 读写头把这个 s 改写成 s' , 并向右移一格;
 $\delta(q, s) = (s', L, q')$ 的含义类似, 只是读写头向左移一格; 若 $\delta(q, s)$ 没有定义, 则 M 停机.

实例



δ	0	1	B
$\rightarrow q_0$	$(0, R, q_0)$	$(1, R, q_0)$	(B, L, q_1)
q_1	(B, L, q_2)	$(1, R, q_0)$	(B, R, q_0)
q_2	(B, R, q_3)	—	—
$*q_3$	—	—	—



格局: 带的内容, 当前的状态和读写头扫视的方格

$$\sigma = \alpha q \beta, \text{ 其中 } \alpha, \beta \in \Gamma^*, q \in Q$$

初始格局 $\sigma_0 = q_0 w$, 其中 $w \in \Sigma^*$ 是输入字符串

接受格局 $\sigma = \alpha q \beta : q \in A$

停机格局 $\sigma = \alpha q s \beta : \delta(q, s) \text{ 没有定义}$

$\sigma_1 \vdash \sigma_2$: 从 σ_1 经过一步能够到达 σ_2 , 称 σ_2 是 σ_1 的**后继**

$\sigma_1 \vdash^* \sigma_2$: 从 σ_1 经过若干步能够到达 σ_2

计算: 一个有穷的或无穷的格局序列, 序列中的每一个格局都是前一个格局的后继.

$\forall w \in \Sigma^*, M$ 从 $\sigma_0 = q_0 w$ 开始的计算有3种可能:

- (1) 停机在接受格局, 即计算为 $\sigma_0, \sigma_1, \dots, \sigma_n$, 其中 σ_n 是接受的停机格局;
- (2) 停机在非接受格局, 即计算为 $\sigma_0, \sigma_1, \dots, \sigma_n$, 其中 σ_n 是非接受的停机格局;
- (3) 永不停机, 即计算为 $\sigma_0, \sigma_1, \dots, \sigma_n, \dots$

图灵机证明了存在不可计算的问题，即无论怎样设计图灵机，都无法得到答案的问题。比如停机问题（halting problem），就是判断**给定一个图灵机和一个输入，它是否会停止运行**。图灵证明了这个问题是不可计算的，因为如果**存在一个能解决这个问题**的图灵机，就会导致逻辑矛盾。

图灵机定义了可计算性（computability）和复杂度（complexity）的标准。比如 P 问题就是指能被图灵机在多项式时间内解决的问题， NP 问题就是指能在多项式时间内被图灵机验证答案的问题。

- 给定一个算法和相应的输入，判定这个算法是否能够完成？
(或进入死循环，永远不能完成?)
- 程序正确性证明，包括了给一段程序，判断这段程序是否会进入死循环
- 由于算法都可以用图灵机描述，所以问题转变为，给定一个图灵机的相应的输入，判定是否停机。
- 对于特定的算法，大概是可以判定的，但是否存在对一般算法进行判定的方法？（该方法本身也是一个算法）

答案是否定的，不存在这样的算法，停机问题是**不可计算问题**

定义 $\forall w \in \Sigma^*$, 如果 M 从 $\sigma_0 = q_0 w$ 开始的计算停机在接受格局, 则称 M 接受输入串 w . M 接受的语言 $L(M)$ 是 M 接受的所有输入串, 即 $L(M) = \{w \in \Sigma^* \mid M \text{ 接受 } w\}$.

实例 (续) M 关于输入 $w=10100$ 的计算:

$q_0 10100B \vdash 1q_0 0100B \vdash 10q_0 100B \vdash 101q_0 00B \vdash 1010q_0 0B$
 $\vdash 10100q_0 B \vdash 1010q_1 0B \vdash 101q_2 0BB \vdash 101Bq_3 BB$

由于停机在接受格局, 故 M 接受 10100.

$$L(M) = \{w00 \mid w \in \{0,1\}^*\}$$



定义 能被图灵机接受的语言称作**递归可枚举的**,
记作 $r.e.$ 也被称为可计算语言

定理 语言 L 是 $r.e.$ 当且仅当 L 是 0 型语言.

图灵机与 0 型文法是等价的

Σ 上的 m 元部分字函数: $(\Sigma^*)^m$ 的某个子集到 Σ^* 的部分函数

TM M 计算的 m 元部分字函数 f : 设输入字母表为 Σ ,

$\forall x_1, \dots, x_m \in \Sigma^*$, 如果 M 从初始格局 $\sigma_0 = q_0 x_1 B \dots x_m B$ 开始的计算停机(不管是否停机在接受状态), 从停机时带的内容中删去 Σ 以外的字符, 得到字符串 y , 则 $f(x_1, x_2, \dots, x_m) = y$;

如果 M 从初始格局 σ_0 开始的计算永不停机, 则 $f(x_1, x_2, \dots, x_m)$ 没有定义, 记作 $f(x_1, x_2, \dots, x_m) \uparrow$.

实例例 (续) M 计算函数: $\forall x \in \{0, 1\}^*$,

$$f(x) = \begin{cases} w, & \text{若 } x = w00 \\ w1, & \text{若 } x = w10 \\ \uparrow, & \text{其他情况} \end{cases}$$

数论函数: 自然数集合 N 上的函数

N 上的 m 元部分函数

N 上的 m 元全函数: 在 N^m 的每一点都有定义

例如 $x+y$ 是全函数, $x-y$ 是部分函数, 当 $x < y$ 时, $x-y \uparrow$

一进制表示: 用 1^x 表示自然数 x

例如 111表示3, 空串 ε 表示0

数论函数的一进制表示: 字母表 $\{1\}$ 上的字函数, 用一进制表示自然数

例如 $x+y$ 可表成 $f(1^x, 1^y) = 1^{x+y}$

定义 设 f 是 N 上的 m 元部分函数, 如果图灵机 M 计算 f 的一进制表示, 即 M 的输入字母表为 $\{1\}, \forall x_1, \dots, x_m \in N$, 从初始格局 $\sigma_0 = q_0 1^{x_1} B 1^{x_2} B \dots 1^{x_m} B$ 开始, 若 $f(x_1, \dots, x_m) = y$, 则 M 的计算停机, 且停机时带的内容(不计 $\{1\}$ 以外的字符)为 1^y ; 若 $f(x_1, \dots, x_m) \uparrow$, 则 M 永不停机, 那么称 M 以一进制方式计算 f .

定义 图灵机 M 以一进制方式计算的 N 上的 m 元部分函数称作**部分递归函数**, 或**部分可计算函数**; 部分递归的全函数称作**递归函数**, 或**可计算函数**.

实例 (续) M 以一进制方式计算

$$f(x) = \begin{cases} x / 4, & \text{若 } x \text{ 被 } 4 \text{ 整除} \\ x / 2, & \text{若 } x \text{ 被 } 2 \text{ 整除, 但不被 } 4 \text{ 整除} \\ \uparrow, & \text{否则} \end{cases}$$

这是一个部分递归函数.

- 图灵机虽然是理论模型，但它奠定了现代计算机设计的基础。
- 图灵机为现代复杂度理论（如 P vs NP 问题）提供了基础语言。
- Python、C++、Java 等现代语言都是图灵完备的，能模拟图灵机的任意算法。

形式语言与形式文法

基本概念：形式语言的基本构成、形式文法及其分类

应用：求形式文法生成的语言

有穷自动机 (FA)

基本概念：自动机的表示，五元组中每个元素的含义、状态转移图、状态转移表、自动机之间的等价性、自动机与形式文法的等价性

应用：简单的自动机之间的转换

图灵机

基本概念：图灵机的基本模型（七元组的含义）、图灵机的计算、格局、停机